CLARENT INSTITUTE

Javascript Array & String Methods, Dialog box, Storage, Geolocation

What are Array Methods?

JavaScript provides built-in methods to manipulate arrays effectively. Commonly used methods:

- `forEach`: Iterate over elements
- `map`: Transform elements
- `filter`: Select specific elements

forEach

- Used to iterate through array elements.
- Executes a provided function once for each array element.
- Not suitable for chaining or returning a new array.

Syntax:

```
array.forEach((element, index, array) => {
      // Your code here
    });
```

Example:

```
    const numbers = [1, 2, 3];
    numbers.forEach((num) => console.log(num));
    // Output: 1, 2, 3
```

map

- Creates a new array by applying a function to each element.
- Original array remains unchanged.
- Transforming data (e.g., converting values).

Syntax:

```
const newArray = array.map((element, index, array) => {
  return element * 2; // Example
});
```

Example:

```
const numbers = [1, 2, 3];
const doubled = numbers.map((num) => num * 2);
console.log(doubled);
// Output: [2, 4, 6]
```

filter

- Creates a new array with elements that pass a test.
- Returns only elements that satisfy the condition.
- Extracting subsets of data (e.g., filtering users by age or status).

Syntax:

```
const filteredArray = array.filter((element, index, array) => {
  return element > 2; // Example
});
```

Example:

```
const numbers = [1, 2, 3, 4];
const greaterThanTwo = numbers.filter((num) => num > 2);
console.log(greaterThanTwo);
// Output: [3, 4]
```

foreach, map and filter

Method	Purpose	Returns a New Array
`forEach`	Iterates	No
`map`	Transforms elements	Yes
`filter`	Selects elements	Yes

String method substring()

Extracts a part of a string between two indices.
Syntax: `str.substring(start, end)`
Features:

- `start`: Start index (inclusive).
- `end`: End index (exclusive, optional).
- Automatically swaps `start` and `end` if `start > end`.
- Does not accept negative indices.

Example:

javascript
Copy code

```
let str = "JavaScript";
console.log(str.substring(0, 4)); // "Java"
console.log(str.substring(4));    // "Script"
console.log(str.substring(5, 2)); // "vaS" (start > end)
```

String method substr()

- Extracts a substring starting at a specified index and for a given length.
- Syntax: `str.substr(start, length)`
- **Features**:
  - `start`: Starting index (can be negative).
  - `length`: Number of characters to extract.

Example:

```
let str = "JavaScript";
console.log(str.substr(4, 6));   // "Script"
console.log(str.substr(-6, 3)); // "Scr"
console.log(str.substr(4));     // "Script" (rest of string)
```

String method slice()

- Extracts a part of a string using indices.
- Syntax: `str.slice(start, end)`
- **Features**:
  - Works like `substring`, but supports negative indices.
  - Negative indices count from the end of the string.
  - `end` is exclusive (optional).

Example:

```
let str = "JavaScript";
console.log(str.slice(0, 4));  // "Java"
console.log(str.slice(-6));   // "Script"
console.log(str.slice(4, -3)); // "Scr"
```

String method split()

- Splits a string into an array based on a delimiter.
- Syntax: `str.split(separator, limit)`
- **Features**:
  - `separator`: Character(s) used to split the string.
  - `limit`: Maximum number of splits (optional).

Example:

```
let str = "apple,banana,orange";
console.log(str.split(",")); // ["apple", "banana", "orange"]
console.log(str.split(",", 2)); // ["apple", "banana"]
```

String method join()

- Combines elements of an array into a single string.
- Syntax: `arr.join(separator)`
- **Features**:
  - `separator`: Character(s) to join array elements (default is a comma `,`).
  - Opposite of `split`, which separates a string into an array.

Example:

```
let fruits = ["apple", "banana", "orange"];
console.log(fruits.join(", ")); // "apple, banana, orange"
console.log(fruits.join(" - ")); // "apple - banana - orange"
```

Dialog Boxes in JavaScript

Method	Purpose	Returns value
`alert()`	Displays a simple message to the user.	No
`confirm()`	Asks the user to confirm or cancel an action.	Yes
`prompt()`	Captures input from the user.	Yes

Example :

```
alert("This is an alert!");
let confirmed = confirm("Do you agree?");
//Returns true if "OK" is clicked, otherwise false.
console.log(result ? "Confirmed" : "Cancelled");
let name = prompt("What is your name?");
//Asks the user for input. Returns the entered value or null if canceled.
console.log(`User entered: ${userInput}`);
```

Web Storage API

Types:

- `localStorage`: Data persists until explicitly deleted.
- `sessionStorage`: Data lasts for the browser session.

Stores key-value pairs.

Example :

```
// Local Storage
localStorage.setItem("username", "Jagdish Patel");
console.log(localStorage.getItem("username"));

// Session Storage
sessionStorage.setItem("isLoggedIn", "true");
console.log(sessionStorage.getItem("isLoggedIn"));
```

Geolocation API

- Allows access to the user's location (with permission).
- Useful for maps, location-based services, etc.

Methods:

1. `getCurrentPosition(successCallback, errorCallback)`
2. `watchPosition(successCallback, errorCallback)`

Example:

```
navigator.geolocation.getCurrentPosition(
  position => {
    console.log(`Latitude: ${position.coords.latitude}`);
    console.log(`Longitude: ${position.coords.longitude}`);
  },
  error => console.error(error.message)
);
```

Variable Declarations in JavaScript

Variables are containers for storing data.
Declared using `var`, `let`, or `const`.
ES6 introduced `let` and `const` to improve scoping and reduce errors.

Feature	`var`	`let`	`const`
Scope	Function-scoped	Block-scoped	Block-scoped
Reassignment	Allowed	Allowed	Not allowed
Redeclaration	Allowed	Not allowed	Not allowed

When to Use `var`, `let`, and `const`

- Use `const` by default.
- Use `let` if reassignment is necessary.
- Avoid `var` in modern JavaScript; reserved for legacy code.